ECOLE NORMALE SUPÉRIURE DE L'ENSEIGNEMENT
TECHNIQUE DE MOHAMMEDIA
UNIVERSITÉ HASSAN II DE CASABLANCA

ADRIA
BUSINESS & TECHNOLOGY

Épreuve Technique de Sélection

Architecture micro-services, IA et développement full-stack

Rapport – Plateforme bancaire distribuée

Réalisé par : Yousef ELHAID
Filière : GLSID – ENSET Mohammedia
Dépôt GitHub : https://github.com/ELHAIDYousef/elhaid-yousef-test-adria-pfe-27

Année universitaire 2026 – 2027

Table des matières

1 Introduction **2**
1.1 Contexte . . . 2
1.2 Périmètre . . . 2
1.3 Stack technique . . . 2

2 Partie A – Préparation et consignes **3**
2.1 Dépôt GitHub . . . 3
2.2 Structure du projet Maven . . . 3

3 Partie B – Conception et architecture **4**
3.1 Diagramme de classes . . . 4
3.2 Architecture . . . 5

4 Partie C – Micro-services fonctionnels **6**
4.1 Account-Service . . . 6
4.1.1 Connecteur REST . . . 6
4.1.2 Connecteur GraphQL . . . 9
4.1.3 Connecteur MCP . . . 10
4.2 Transaction-Service . . . 11
4.2.1 Connecteur REST . . . 11
4.2.2 Connecteur GraphQL . . . 12
4.2.3 Connecteur MCP . . . 13
4.3 Communication OpenFeign et tolérance aux pannes (Resilience4J) . . . 13

5 Partie D – Micro-services techniques **15**
5.1 Eureka Discovery Service . . . 15
5.2 Spring Cloud Gateway . . . 15

6 Partie E – Chatbot agentique **17**

1 Introduction

1.1 Contexte

Une banque souhaite une plateforme pour consulter les comptes bancaires, enregistrer les transactions et publier des événements métier. Un assistant conversationnel permet d'interroger ces données depuis une interface web ou via Telegram.

1.2 Périmètre

Ce rapport couvre les parties **A à G** de l'épreuve. Les parties **H** (IA Python) et **I** (DevOps) sont traitées selon le temps disponible.

1.3 Stack technique

Domaine	Technologies
Langage	Java 21
Framework	Spring Boot 4.1.2
Micro-services	Spring Cloud : Gateway, Eureka, Config Server, OpenFeign, Resilience4J
Exposition	REST, GraphQL, MCP
IA	Spring AI (agent connecté aux serveurs MCP)
Messagerie	Apache Kafka
Client web	Angular
Outils	Maven, Docker, Git / GitHub

TABLE 1 – Technologies utilisées

2 Partie A – Préparation et consignes

2.1 Dépôt GitHub

— Dépôt `elhaid-yousef-test-adria-pfe-27`, **privé** pendant l'épreuve puis rendu **public**.

— Une seule branche (`main`) et un commit environ toutes les 15 minutes.

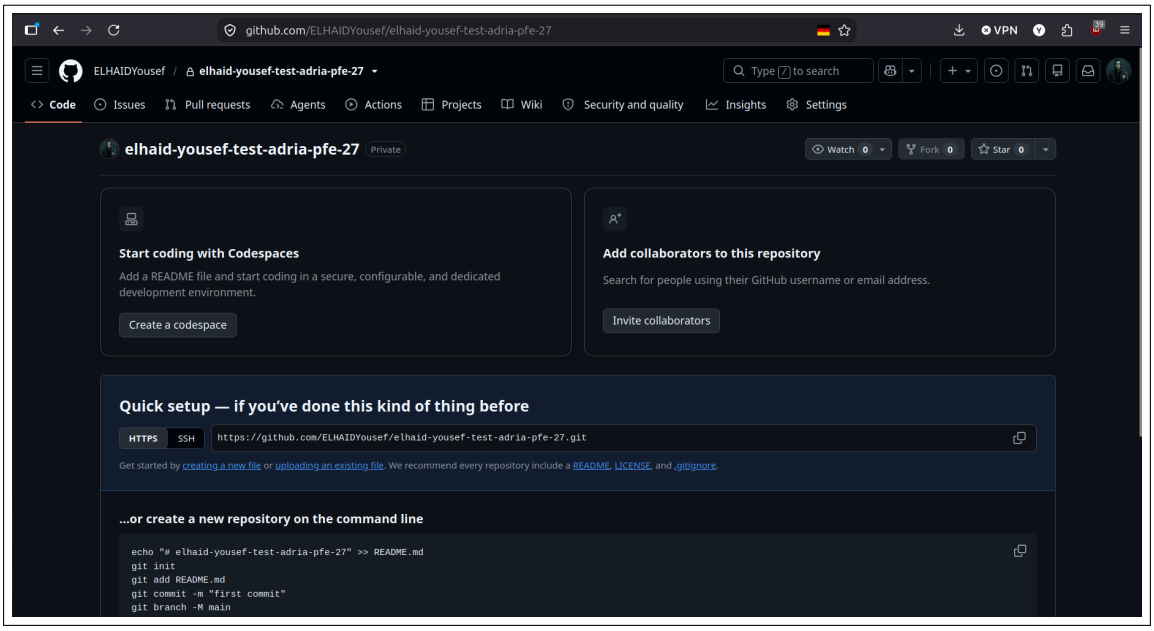


FIGURE 1 – Dépôt GitHub privé

2.2 Structure du projet Maven

Un `pom.xml` racine regroupe les micro-services sous forme de **modules**. Il sert uniquement d'**agrégateur** : chaque micro-service reste indépendant et garde son propre parent Spring Boot.

```xml
<groupId>ma.enset</groupId>
<artifactId>elhaid-yousef-test-adria-pfe-27</artifactId>
<version>1.0.0-SNAPSHOT</version>
<packaging>pom</packaging>

<modules>
    <!-- un module par micro-service -->
</modules>
```

Code 1 – `pom.xml` racine

```
elhaid-yousef-test-adria-pfe-27/
|-- pom.xml
|-- .gitignore
`-- docs/screenshots/
```

Code 2 – Arborescence du dépôt

3 Partie B – Conception et architecture

3.1 Diagramme de classes

Les deux micro-services métier suivent la même découpe en couches (`domain`, `repository`, `service`, `web/rest`, `web/graphql`, `mcp`) : les contrôleurs REST, GraphQL et MCP appellent la **même couche service**.

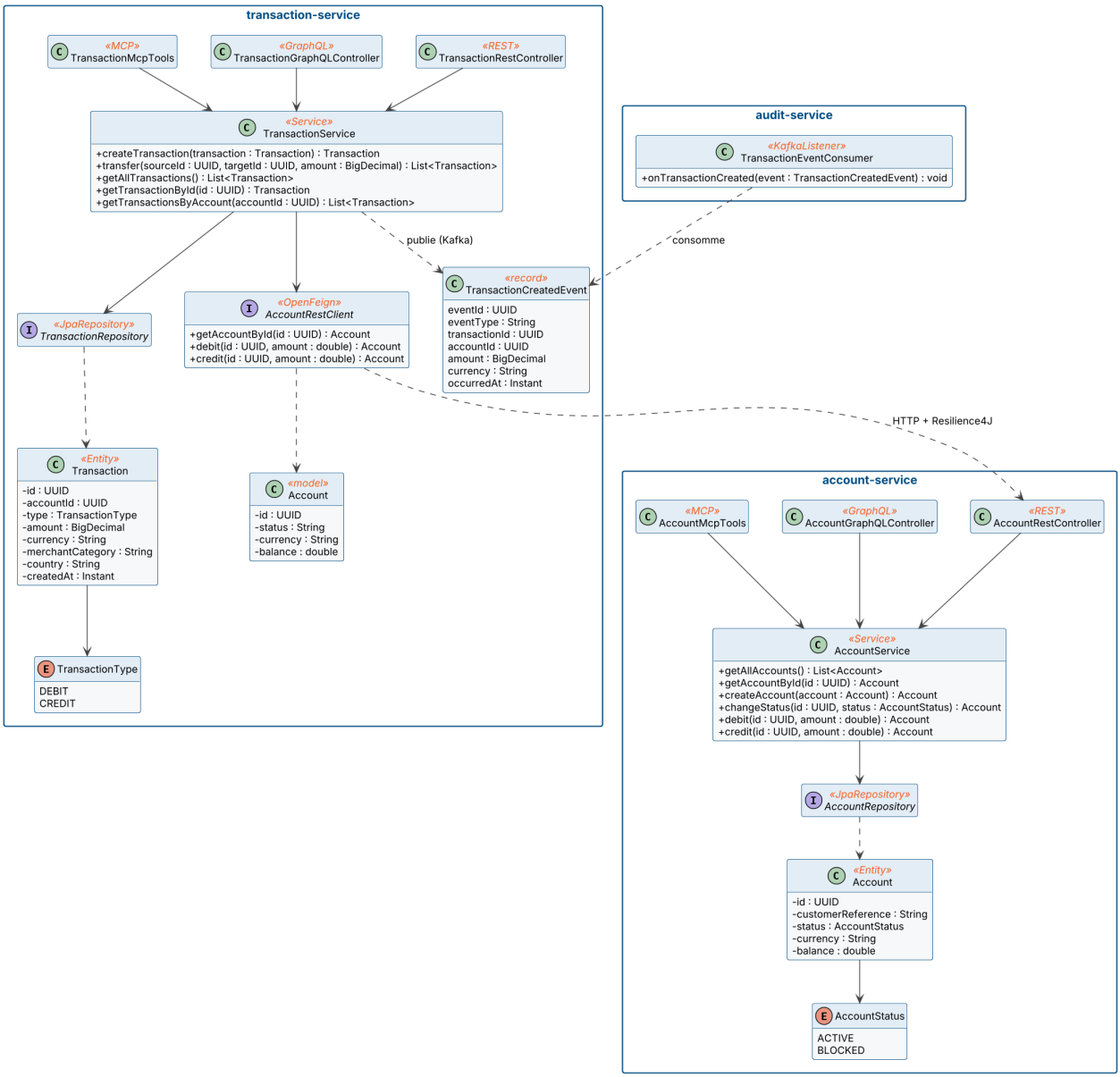


FIGURE 2 – Diagramme de classes

Choix de conception :

— Le modèle `Transaction` est enrichi d'un champ `type` (`DEBIT` / `CREDIT`). Un virement crée un débit sur le compte source et un crédit sur le compte destination, avec un montant toujours positif.

— `transaction-service` ne stocke pas les comptes : il les lit via OpenFeign (classe `Account` non persistée) et demande à `account-service` de débiter ou créditer.

— Après chaque enregistrement, un événement `TransactionCreatedEvent` est publié dans Kafka.

3.2 Architecture

Les clients passent par la **gateway**, qui localise les services via **Eureka**. Le **chatbot** accède aux services métier via MCP. Chaque transaction est publiée dans **Kafka** puis consommée par `audit-service`.

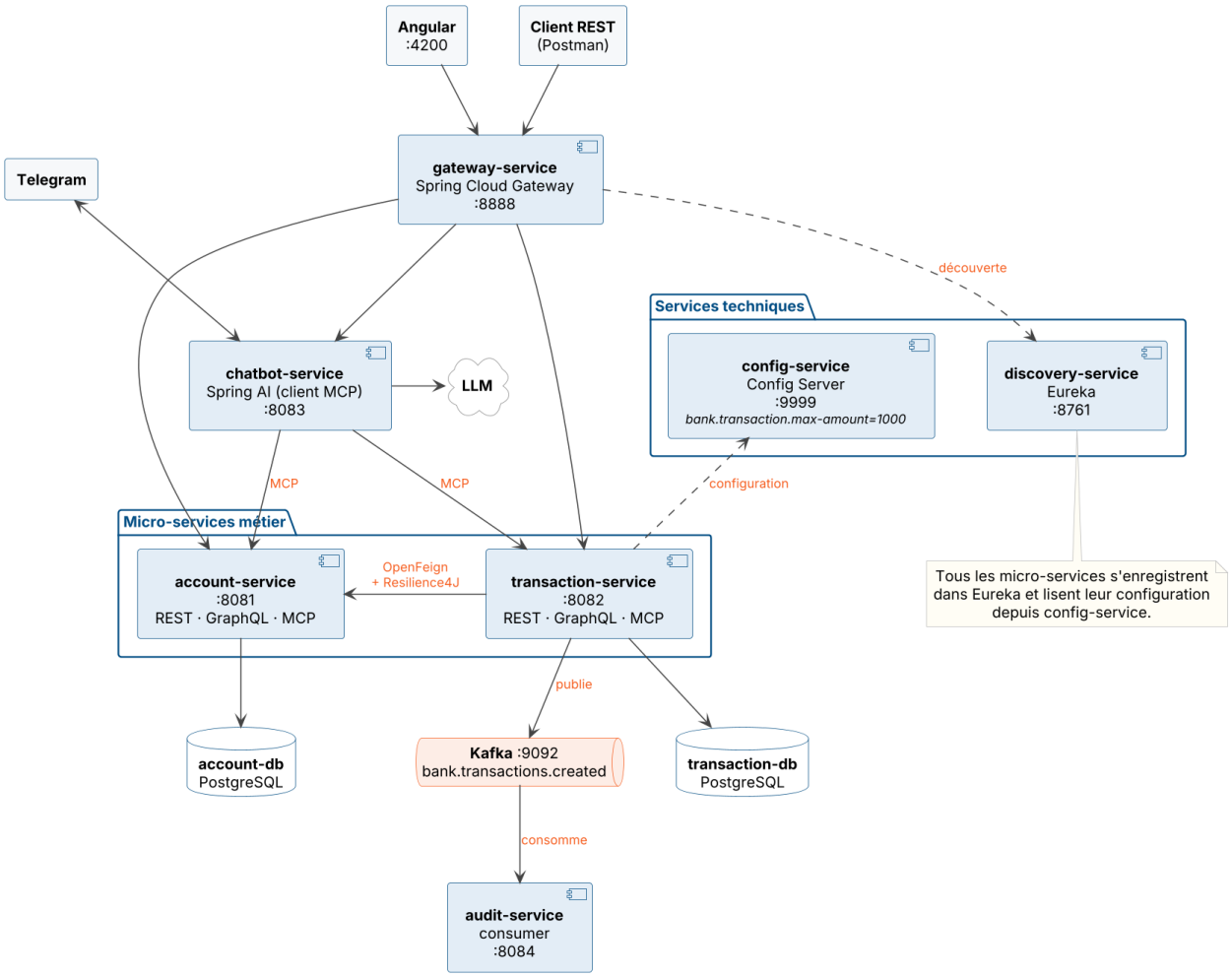


FIGURE 3 – Architecture de la plateforme

4 Partie C – Micro-services fonctionnels

4.1 Account-Service

Le service gère les comptes bancaires. Il suit la séparation en couches imposée : les connecteurs REST, GraphQL et MCP appellent tous `AccountService`.

```
ma.enset.accountservice
|-- domain/         Account, AccountStatus
|-- repository/     AccountRepository
|-- service/        AccountService, AccountNotFoundException, BusinessRuleException
|-- web/rest/       AccountRestController, RestExceptionHandler
|-- web/graphql/    AccountGraphQLController
`-- mcp/            AccountMcpTools
```

Code 3 – Structure de `account-service`

Règles métier (couche service) :

— un débit ou un crédit exige un compte **existant** et **actif** et un montant strictement positif ;

— un débit exige un **solde suffisant** ;

— les erreurs renvoient un `ProblemDetail` : **404** (compte introuvable) ou **400** (règle non respectée).

```java
public Account debit(UUID id, double amount) {
    Account account = getActiveAccount(id);
    checkPositive(amount);
    if (account.getBalance() < amount) {
        throw new BusinessRuleException("Insufficient balance on account " + id);
    }
    account.setBalance(account.getBalance() - amount);
    return accountRepository.save(account);
}
```

Code 4 – Règle de débit dans `AccountService`

4.1.1 Connecteur REST

Méthode	URL	Rôle
GET	`/api/accounts`	Liste des comptes
GET	`/api/accounts/{id}`	Détail d'un compte
POST	`/api/accounts`	Création d'un compte
PATCH	`/api/accounts/{id}/status`	Activation / blocage
POST	`/api/accounts/{id}/debit`	Débit
POST	`/api/accounts/{id}/credit`	Crédit

TABLE 2 – API REST de `account-service`

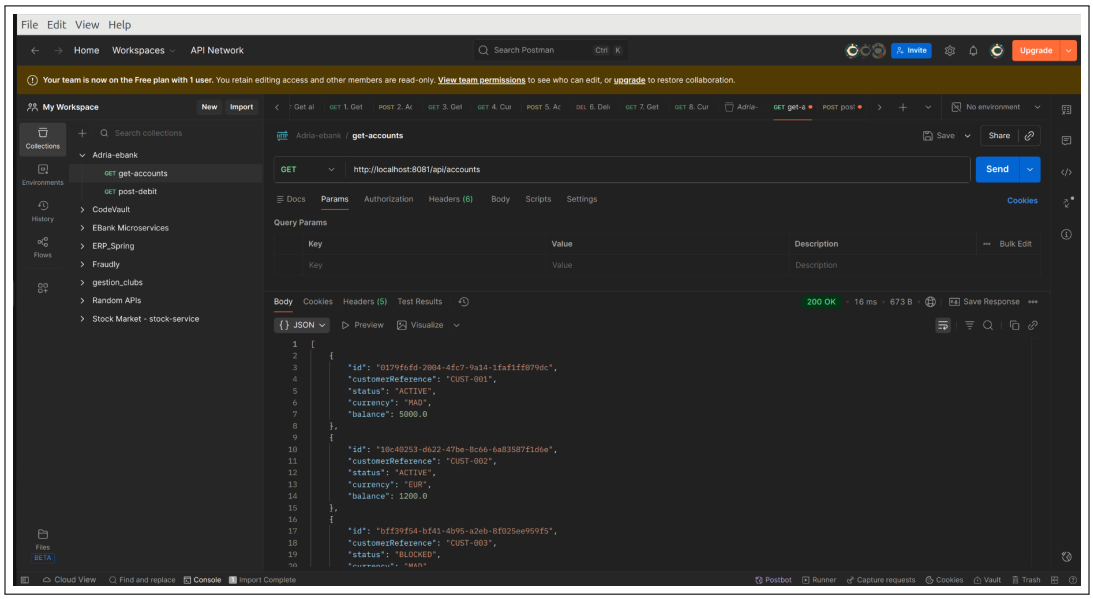


FIGURE 4 – Liste des comptes (données de test)

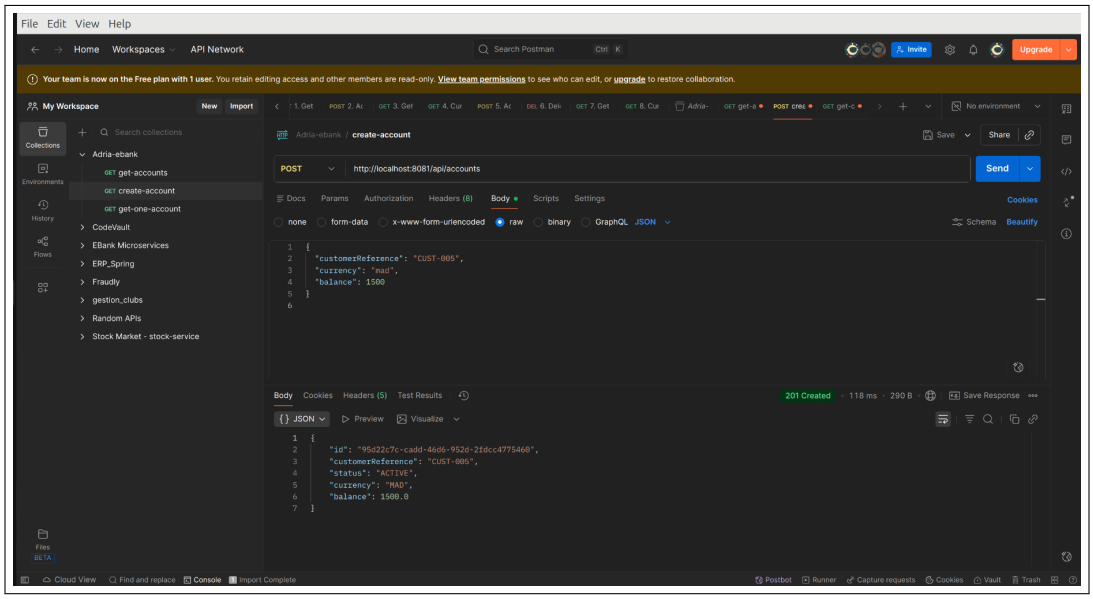


FIGURE 5 – Création d'un compte

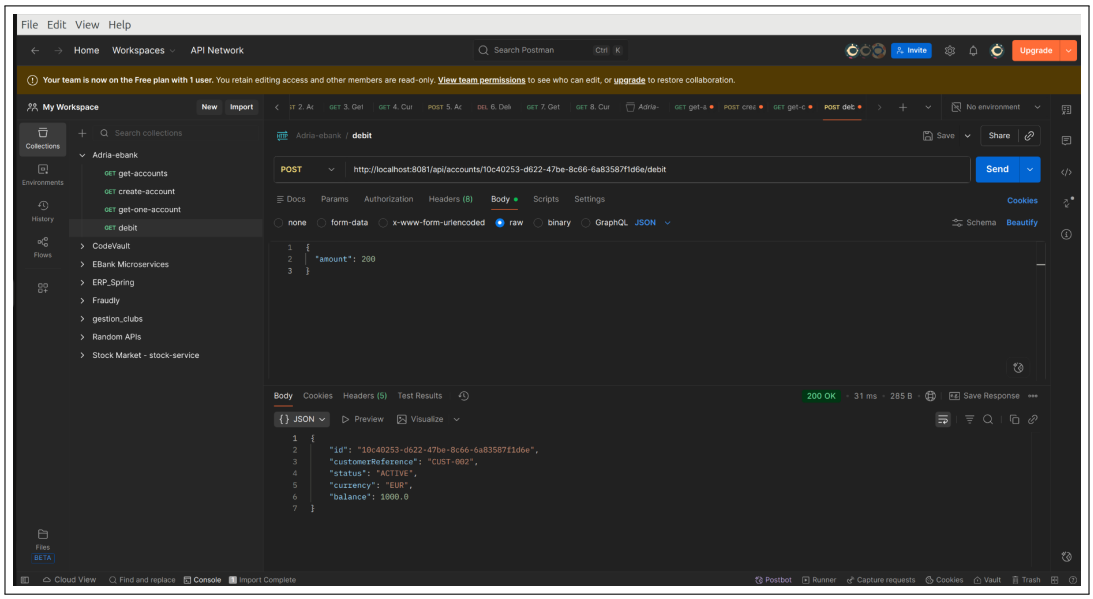


FIGURE 6 – Débit de 200 sur un compte actif

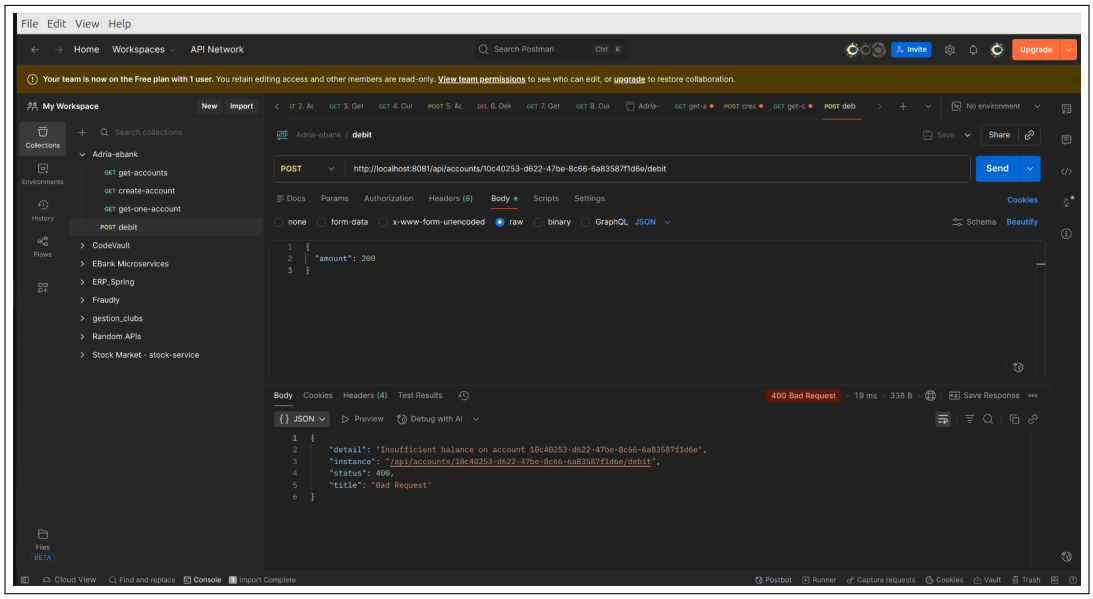


FIGURE 7 – Erreur : solde insuffisant

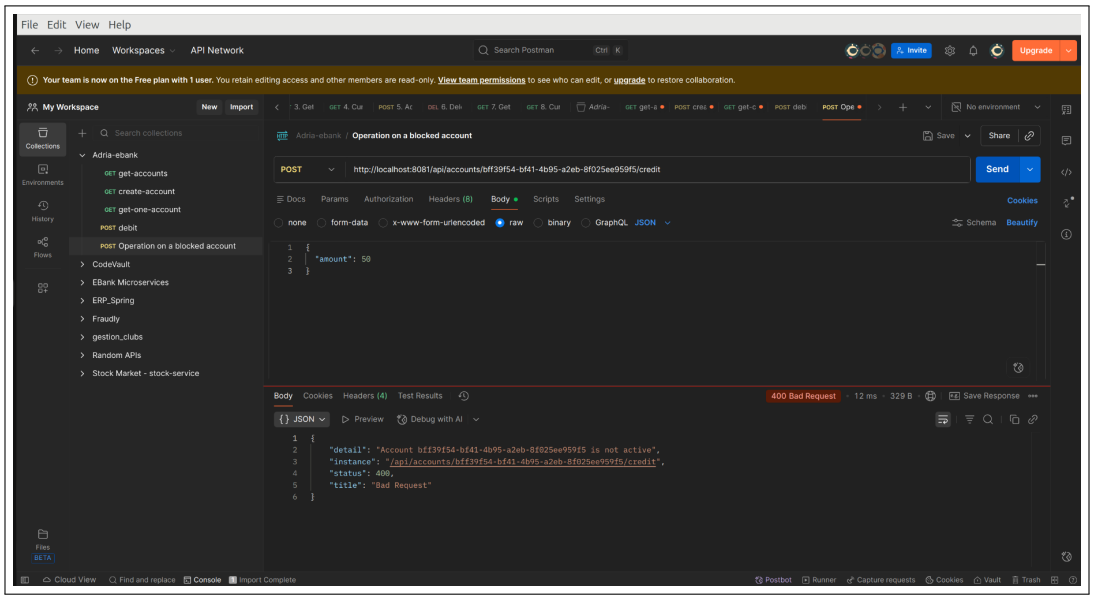


FIGURE 8 – Erreur : opération sur un compte bloqué

4.1.2 Connecteur GraphQL

```
type Query {
    accounts: [Account]
    accountById(id: ID!): Account
}

type Mutation {
    createAccount(account: AccountInput!): Account
    changeStatus(id: ID!, status: AccountStatus!): Account
    debit(id: ID!, amount: Float!): Account
    credit(id: ID!, amount: Float!): Account
}
```

Code 5 – Extrait du schéma `schema.graphqls`

Le client choisit les champs retournés. Les exceptions métier sont converties en erreurs GraphQL (`NOT_FOUND`, `BAD_REQUEST`).

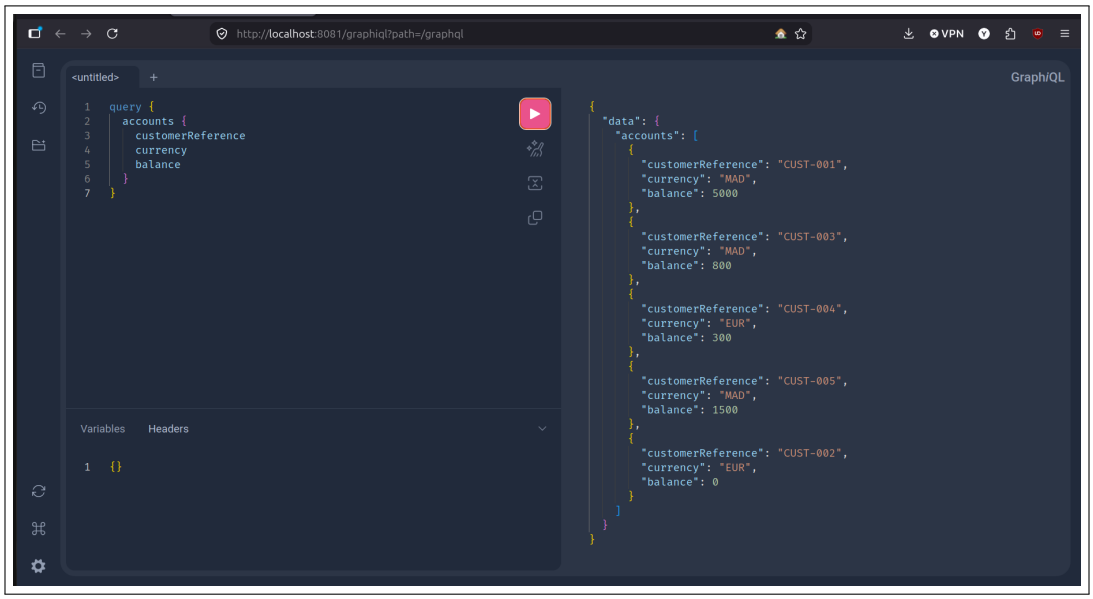


FIGURE 9 – Requête GraphQL : liste des comptes avec 3 champs

4.1.3 Connecteur MCP

Le serveur MCP (`/mcp`) expose des *tools* utilisables par l'agent IA. Le débit et le crédit ne sont **pas** exposés : l'agent doit passer par `transaction-service`, pour que chaque opération soit enregistrée et publiée dans Kafka.

Tool	Rôle
getAllAccounts	Liste des comptes
getAccountById	Détail d'un compte
createAccount	Création d'un compte actif
changeAccountStatus	Activation / blocage d'un compte

TABLE 3 – Tools MCP de `account-service`

```
@McpTool(name = "getAccountById", description = "Get one bank account by its id (UUID)")
public Account getAccountById(@McpToolParam(description = "Account id (UUID)") String id) {
    return accountService.getAccountById(UUID.fromString(id));
}
```

Code 6 – Exemple de tool MCP

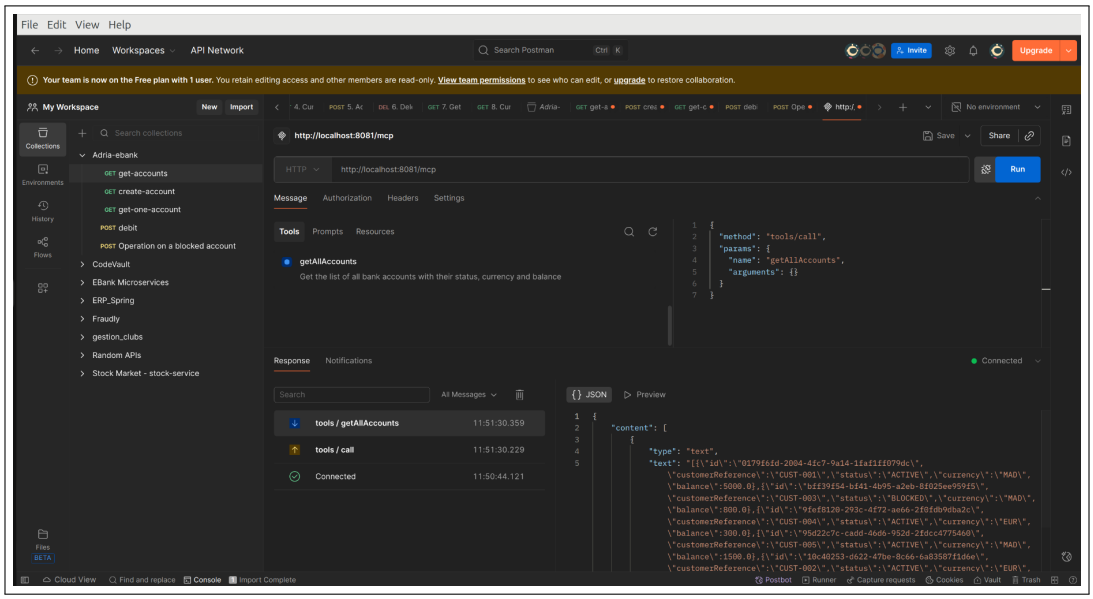


FIGURE 10 – Tools MCP exposés par `account-service`

4.2 Transaction-Service

Le service enregistre et consulte les transactions. Il reprend la même séparation en couches (`domain`, `repository`, `service`, `web/rest`, `web/graphql`, `mcp`).

Choix techniques :

— `amount` en `BigDecimal` (montant exact) et `createdAt` en `Instant` (UTC) ;

— champ `type` (`DEBIT` par défaut, ou `CREDIT`) ;

— le plafond est lu depuis la configuration `bank.transaction.max-amount` (1000), centralisée dans `config-service` en partie D.

```
@Value("${bank.transaction.max-amount}")
private BigDecimal maxAmount;

private void checkAmount(BigDecimal amount) {
    if (amount == null || amount.signum() <= 0) {
        throw new BusinessRuleException("Amount must be strictly positive");
    }
    if (amount.compareTo(maxAmount) > 0) {
        throw new BusinessRuleException("Amount exceeds the maximum allowed: " +
    maxAmount);
    }
}
```

Code 7 – Contrôle du montant dans `TransactionService`

4.2.1 Connecteur REST

Méthode	URL	Rôle
GET	`/api/transactions`	Liste des transactions
GET	`/api/transactions/{id}`	Détail d'une transaction
GET	`/api/transactions/account/{accountId}`	Transactions d'un compte
POST	`/api/transactions`	Création d'une transaction

TABLE 4 – API REST de `transaction-service`

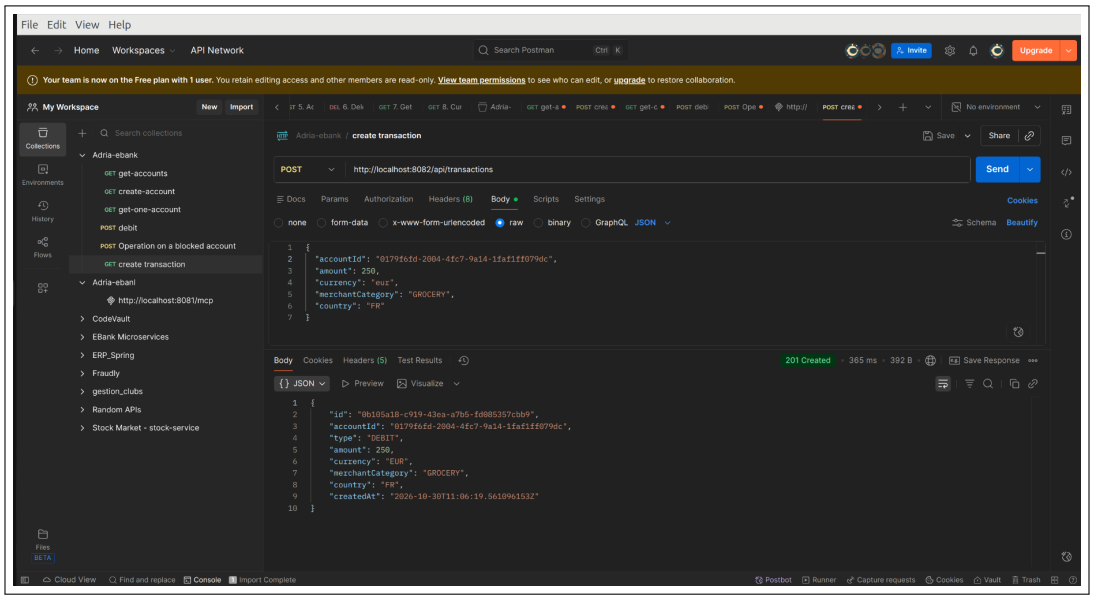


FIGURE 11 – Création d'une transaction

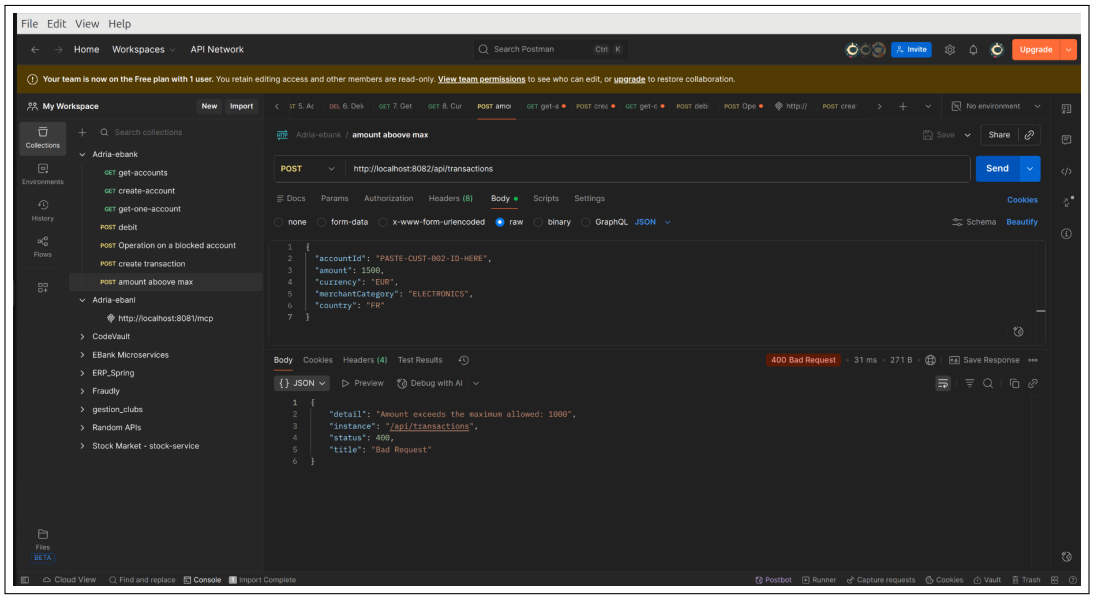


FIGURE 12 – Erreur : montant supérieur au plafond

4.2.2 Connecteur GraphQL

```
type Query {
    transactions: [Transaction]
    transactionById(id: ID!): Transaction
    transactionsByAccount(accountId: ID!): [Transaction]
}

type Mutation {
    createTransaction(transaction: TransactionInput!): Transaction
}
```

Code 8 – Extrait du schéma `schema.graphqls`

4.2.3 Connecteur MCP

Tool	Rôle
getAllTransactions	Liste des transactions
getTransactionById	Détail d'une transaction
getTransactionsByAccount	Transactions d'un compte
createTransaction	Enregistrement d'une transaction (paiement ou crédit)

4.3 Communication OpenFeign et tolérance aux pannes (Resilience4J)

`transaction-service` appelle `account-service` avec un client **OpenFeign** (`AccountRestClient`) pour consulter le compte avant chaque transaction, puis mettre à jour son solde.

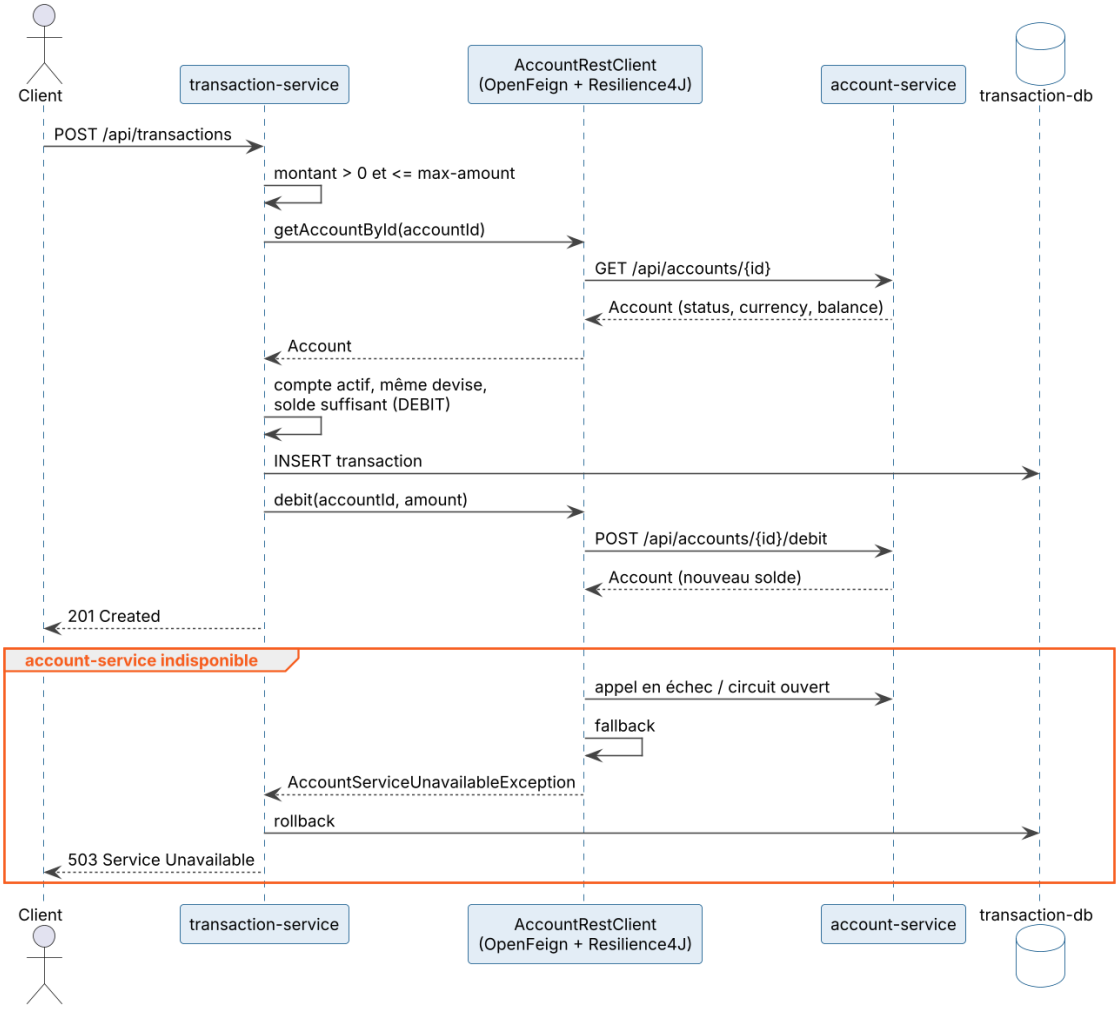


FIGURE 13 – Création d'une transaction

Règles appliquées : montant positif et inférieur au plafond, compte existant et actif, même devise que le compte, solde suffisant pour un débit. Un **virement** crée un débit sur le compte source et un crédit sur le compte destination (même devise).

```
@FeignClient(name = "account-service", url = "${account-service.url}",
        fallbackFactory = AccountRestClientFallbackFactory.class)
public interface AccountRestClient {
    @GetMapping("/api/accounts/{id}")
    Account getAccountById(@PathVariable("id") UUID id);
```

```
6
7       @PostMapping("/api/accounts/{id}/debit")
8       Account debit(@PathVariable("id") UUID id, @RequestBody AmountRequest request);
9
10      @PostMapping("/api/accounts/{id}/credit")
11      Account credit(@PathVariable("id") UUID id, @RequestBody AmountRequest request);
12  }
```

Code 9 – Client OpenFeign avec fallback

Tolérance aux pannes

— Un **circuit breaker** Resilience4J protège chaque appel : au-delà de 50 % d'échecs, il s'ouvre pendant 10 s et les appels sont rejetés immédiatement.

— Le **fallback ne renvoie jamais un faux compte** : il traduit l'erreur (404 pour un compte introuvable, 503 si le service est indisponible) et la transaction est refusée, avec annulation (*rollback*) de l'enregistrement.

5 Partie D – Micro-services techniques

Service	Port	Rôle
discovery-service	8761	Annuaire Eureka : chaque service s'y enregistre
gateway-service	8888	Point d'entrée unique, routage vers les services via Eureka

TABLE 5 – Services techniques

5.1 Eureka Discovery Service

Serveur Eureka (`@EnableEurekaServer`). `account-service`, `transaction-service` et la gateway s'y enregistrent. Le client OpenFeign n'utilise plus d'URL fixe : il appelle `account-service` par son **nom**.

```
@FeignClient(name = "account-service", fallbackFactory =
    AccountRestClientFallbackFactory.class)
public interface AccountRestClient { ... }
```

Code 10 – Client OpenFeign résolu par Eureka

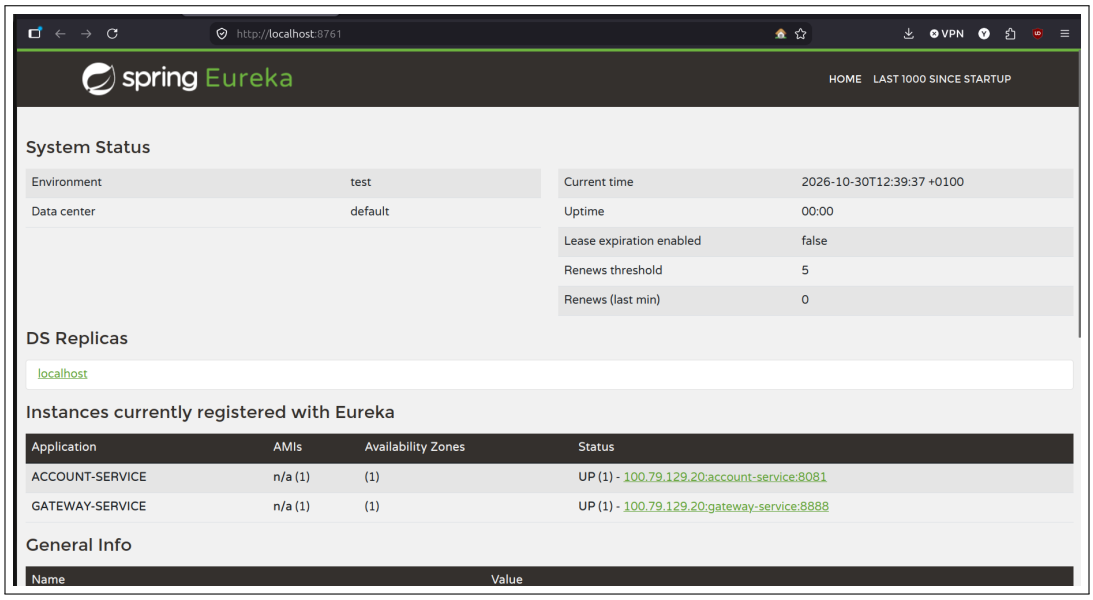


FIGURE 14 – Services enregistrés dans Eureka

5.2 Spring Cloud Gateway

```
routes:
  - id: account-service
    uri: lb://account-service
    predicates:
      - Path=/api/accounts/**
  - id: transaction-service
    uri: lb://transaction-service
    predicates:
      - Path=/api/transactions/**
```

Code 11 – Routes de la gateway

`lb://` signifie que l'adresse réelle est demandée à Eureka (avec répartition de charge).

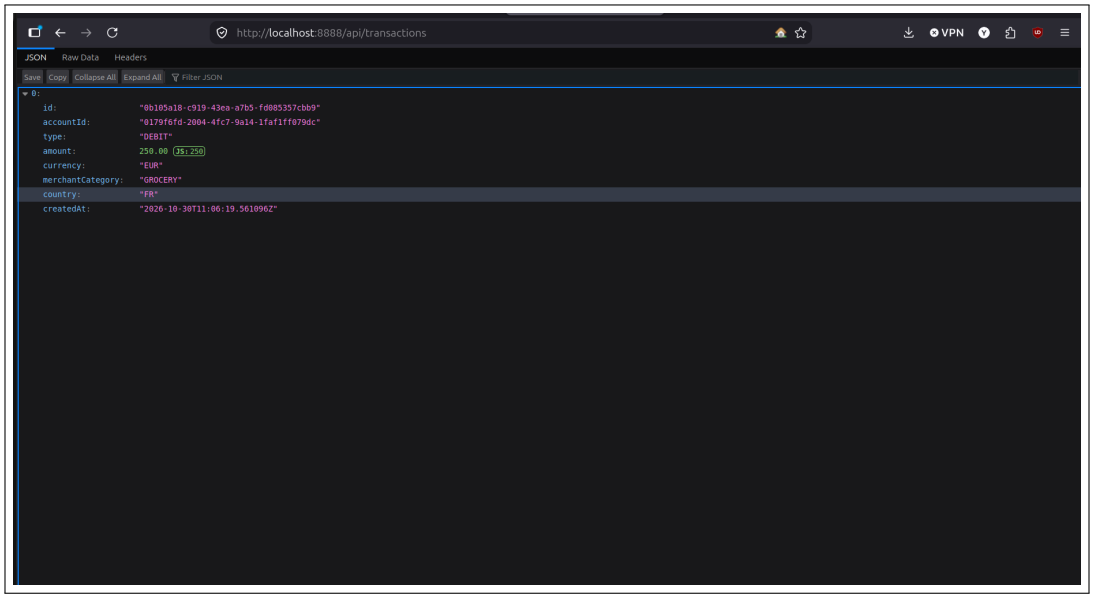


FIGURE 15 – Appel de `account-service` via la gateway

6 Partie E – Chatbot agentique

`chatbot-service` (port 8083) est un agent IA construit avec **Spring AI**. Il agit comme **client MCP** : au démarrage, il se connecte aux serveurs MCP d'`account-service` et de `transaction-service` et récupère leurs *tools*. Le LLM choisit lui-même les tools à appeler pour répondre ou pour effectuer une opération.

```
Utilisateur --> /chat?query=...  --> chatbot-service --> LLM (question + liste des tools)
                                                    |     <-- "appelle getAllAccounts"
                                                    |---> account-service /mcp     (lecture des comptes)
                                                    |---> transaction-service /mcp (paiement, virement)
Utilisateur <-- réponse en langage naturel <-- LLM
```

Code 12 – Fonctionnement de l'agent

```yaml
spring:
  ai:
    mcp:
      client:
        streamable-http:
          connections:
            account:
              url: http://localhost:8081/mcp
            transaction:
              url: http://localhost:8082/mcp
```

Code 13 – Connexions MCP de l'agent